

Data Structure

2D BIT

```
#include <iostream>
#include <vector>
using namespace std;

/**
 * 2D Fenwick Tree implementation.
 * Note that all cell locations are zero-indexed
 * in this implementation.
 */
template <typename T> class BIT2D {
private:
    const int n, m;
    vector<vector<T>> bit;

public:
    BIT2D(int n, int m) : n(n), m(m), bit(n + 1, vector<T>(m + 1)) {}

    /** adds val to the point (r, c) */
    void add(int r, int c, T val) {
        r++, c++;
        for (; r <= n; r += r & -r) {
            for (int i = c; i <= m; i += i & -i) { bit[r][i] += val; }
        }
    }

    /** @returns sum of points with row in [0, r] and column in [0, c] */
    T rect_sum(int r, int c) {
        r++, c++;
        T sum = 0;
        for (; r > 0; r -= r & -r) {
            for (int i = c; i > 0; i -= i & -i) { sum += bit[r][i]; }
        }
        return sum;
    }

    /** @returns sum of points with row in [r1, r2] and column in [c1, c2] */
    T rect_sum(int r1, int c1, int r2, int c2) {
        return rect_sum(r2, c2) - rect_sum(r2, c1 - 1) - rect_sum(r1 - 1, c2)
+
        rect_sum(r1 - 1, c1 - 1);
    }
};
```

```

    }
};

int main() {
    int n, q;
    cin >> n >> q;
    BIT2D<int> bit(n, n);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            char c;
            cin >> c;
            if (c == '*') { bit.add(i, j, 1); }
        }
    }

    for (int i = 0; i < q; i++) {
        int type;
        cin >> type;
        if (type == 1) {
            int r, c;
            cin >> r >> c;
            r--, c--;
            if (bit.rect_sum(r, c, r, c) == 1) {
                bit.add(r, c, -1);
            } else {
                bit.add(r, c, 1);
            }
        } else if (type == 2) {
            int r1, c1, r2, c2;
            cin >> r1 >> c1 >> r2 >> c2;
            r1--, c1--, r2--, c2--;
            cout << bit.rect_sum(r1, c1, r2, c2) << '\n';
        }
    }
}

```

range update

```

/**
 * 2D Fenwick Tree (RURQ) implementation. * API is 0-indexed. Internals are 1-
indexed. * Note: Use long long for T to prevent overflow from coordinate
multiplication. */template <typename T> class BIT2D_RURQ {
private:
    int n, m;
    vector<vector<T>> bit1, bit2, bit3, bit4;

```

```

// Internal 1-indexed point update
void point_add(int r, int c, T val) {
    for (int i = r; i <= n; i += i & -i) {
        for (int j = c; j <= m; j += j & -j) {
            bit1[i][j] += val;
            bit2[i][j] += val * r;
            bit3[i][j] += val * c;
            bit4[i][j] += val * r * c;
        }
    }
}

// Internal 1-indexed prefix sum
T prefix_sum(int r, int c) {
    T sum = 0;
    for (int i = r; i > 0; i -= i & -i) {
        for (int j = c; j > 0; j -= j & -j) {
            sum += bit1[i][j] * (r + 1) * (c + 1)
                - bit2[i][j] * (c + 1)
                - bit3[i][j] * (r + 1)
                + bit4[i][j];
        }
    }
    return sum;
}

public:
    BIT2D_RURQ(int n, int m) : n(n), m(m),
        bit1(n + 1, vector<T>(m + 1, 0)),
        bit2(n + 1, vector<T>(m + 1, 0)),
        bit3(n + 1, vector<T>(m + 1, 0)),
        bit4(n + 1, vector<T>(m + 1, 0)) {}

    /** Adds val to the rectangle bounded by (r1, c1) and (r2, c2) (0-indexed)
    */
    void range_add(int r1, int c1, int r2, int c2, T val) {
        r1++; c1++; r2++; c2++; // Shift to 1-indexed
        point_add(r1, c1, val);
        point_add(r2 + 1, c1, -val);
        point_add(r1, c2 + 1, -val);
        point_add(r2 + 1, c2 + 1, val);
    }

    /** @returns sum of points with row in [r1, r2] and column in [c1, c2] (0-indexed) */

```

```

T rect_sum(int r1, int c1, int r2, int c2) {
    r1++; c1++; r2++; c2++; // Shift to 1-indexed
    return prefix_sum(r2, c2)
        - prefix_sum(r1 - 1, c2)
        - prefix_sum(r2, c1 - 1)
        + prefix_sum(r1 - 1, c1 - 1);
}
};

```

```

## Xor version
#include <iostream>
#include <vector>

using namespace std;

/**
 * 2D Fenwick Tree (Range XOR Update, Range XOR Query).
 * API is 0-indexed. Internals are 1-indexed.
 */
template <typename T> class BIT2D_XOR {
private:
    int n, m;
    vector<vector<T>> bit[2][2];

    void point_xor(int r, int c, T val) {
        int r_par = r % 2;
        int c_par = c % 2;
        for (int i = r; i <= n; i += i & -i) {
            for (int j = c; j <= m; j += j & -j) {
                bit[r_par][c_par][i][j] ^= val;
            }
        }
    }

    T prefix_xor(int r, int c) {
        if (r < 1 || c < 1) return 0;
        T res = 0;
        int r_par = r % 2;
        int c_par = c % 2;
        for (int i = r; i > 0; i -= i & -i) {
            for (int j = c; j > 0; j -= j & -j) {
                res ^= bit[r_par][c_par][i][j];
            }
        }
        return res;
    }
};

```

```

    }

public:
    BIT2D_XOR(int n, int m) : n(n), m(m) {
        for (int i = 0; i < 2; i++) {
            for (int j = 0; j < 2; j++) {
                bit[i][j].assign(n + 1, vector<T>(m + 1, 0));
            }
        }
    }

    /** XORs val into the rectangle bounded by (r1, c1) and (r2, c2) (0-
indexed) */
    void range_xor(int r1, int c1, int r2, int c2, T val) {
        r1++; c1++; r2++; c2++; // Shift to 1-indexed
        point_xor(r1, c1, val);
        point_xor(r2 + 1, c1, val);
        point_xor(r1, c2 + 1, val);
        point_xor(r2 + 1, c2 + 1, val);
    }

    /** @returns XOR sum of points with row in [r1, r2] and column in [c1, c2]
(0-indexed) */
    T rect_xor(int r1, int c1, int r2, int c2) {
        r1++; c1++; r2++; c2++;
        return prefix_xor(r2, c2)
            ^ prefix_xor(r1 - 1, c2)
            ^ prefix_xor(r2, c1 - 1)
            ^ prefix_xor(r1 - 1, c1 - 1);
    }
};

int main() {
    // Fast I/O
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    int n, q;
    cin >> n >> q;
    BIT2D_XOR<int> bit(n, n);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            char c;
            cin >> c;
            if (c == '*') {

```

```

        bit.range_xor(i, j, i, j, 1);
    }
}

for (int i = 0; i < q; i++) {
    int type;
    cin >> type;

    if (type == 1) {
        int r1, c1, r2, c2, val;
        cin >> r1 >> c1 >> r2 >> c2 >> val;
        r1--, c1--, r2--, c2--;
        bit.range_xor(r1, c1, r2, c2, val);
    } else if (type == 2) {
        int r1, c1, r2, c2;
        cin >> r1 >> c1 >> r2 >> c2;
        r1--, c1--, r2--, c2--;
        cout << bit.rect_xor(r1, c1, r2, c2) << '\n';
    }
}

return 0;
}

```

2D segtree

```

#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;
template<typename T>
using orderedset = tree<T, null_type, less<T>, rb_tree_tag,
tree_order_statistics_node_update>;
#define int long long
#define ll long long
#define ld long double
#define endl '\n'
#define wady \
    ios_base::sync_with_stdio(0); \
    cin.tie(0); \
    cout.tie(0);

```

```

void files() {
#ifdef ONLINE_JUDGE
    freopen("in.txt", "r", stdin);
    freopen("out.txt", "w", stdout);
#endif
}

struct _2dsegtree {
    int n, m;

    struct Node {
        int val;
    };

    Node skip;
    vector<Node> tree;

    _2dsegtree(int n, int m, vector<vector<int> > &v) {
        this->n = n;
        this->m = m;
        skip.val = 1e15;
        tree.resize((n * m) << 4, skip);
        build(1, 0, n - 1, 0, m - 1, v);
    }

    Node single(int x) {
        Node r;
        r.val = x;
        return r;
    }

    Node merge(Node a, Node b) {
        Node ret;
        ret.val = min(a.val, b.val);
        return ret;
    }

    void build(int u, int r1, int r2, int c1, int c2, vector<vector<int> > &a)
    {
        if (r1 > r2 || c1 > c2) return;
        if (r1 == r2 && c1 == c2) return void(tree[u] = single(a[r1][c1]));
        int mid_r = r1 + r2 >> 1;
        int mid_c = c1 + c2 >> 1;
        build((u << 2) - 2, r1, mid_r, c1, mid_c, a);
        build((u << 2) - 1, mid_r + 1, r2, c1, mid_c, a);
        build((u << 2), r1, mid_r, mid_c + 1, c2, a);
    }
}

```

```

    build((u << 2) | 1, mid_r + 1, r2, mid_c + 1, c2, a);

    tree[u] = merge(
        merge(tree[(u << 2) - 2], tree[(u << 2) - 1]),
        merge(tree[(u << 2)], tree[(u << 2) | 1])
    );
}

void update(int u, int r1, int r2, int c1, int c2, int x, int y, int val)
{
    if (r1 > r2 || c1 > c2 || x < r1 || x > r2 || y < c1 || y > c2)
return;
    if (r1 == r2 && c1 == c2) return void(tree[u] = single(val));

    int mid_r = r1 + r2 >> 1;
    int mid_c = c1 + c2 >> 1;

    update((u << 2) - 2, r1, mid_r, c1, mid_c, x, y, val);
    update((u << 2) - 1, mid_r + 1, r2, c1, mid_c, x, y, val);
    update((u << 2), r1, mid_r, mid_c + 1, c2, x, y, val);
    update((u << 2) | 1, mid_r + 1, r2, mid_c + 1, c2, x, y, val);

    tree[u] = merge(
        merge(tree[(u << 2) - 2], tree[(u << 2) - 1]),
        merge(tree[(u << 2)], tree[(u << 2) | 1])
    );
}

Node query(int u, int r1, int r2, int c1, int c2, int qr1, int qr2, int
qc1, int qc2) {
    if (r1 > r2 || c1 > c2 || r1 > qr2 || r2 < qr1 || c1 > qc2 || c2 <
qc1) return skip;
    if (r1 >= qr1 && r2 <= qr2 && c1 >= qc1 && c2 <= qc2) return tree[u];

    int mid_r = r1 + r2 >> 1;
    int mid_c = c1 + c2 >> 1;

    return merge(
        merge(query((u << 2) - 2, r1, mid_r, c1, mid_c, qr1, qr2, qc1,
qc2),
            query((u << 2) - 1, mid_r + 1, r2, c1, mid_c, qr1, qr2, qc1,
qc2)),
        merge(query((u << 2), r1, mid_r, mid_c + 1, c2, qr1, qr2, qc1,
qc2),
            query((u << 2) | 1, mid_r + 1, r2, mid_c + 1, c2, qr1, qr2,
qc1, qc2))
    );
}

```

```

    );
}

void update(int x, int y, int val) {
    --x, --y;
    update(1, 0, n - 1, 0, m - 1, x, y, val);
}

Node query(int r1, int c1, int r2, int c2) {
    --r1, --c1, --r2, --c2;
    return query(1, 0, n - 1, 0, m - 1, r1, r2, c1, c2);
}

};

void solve(int tc) {
    int n, m, p;
    cin >> n >> m >> p;
    vector<vector<int>> > dp(n + 2, vector<int>(m + 2, 1e13));
    _2dsegtree tr1(n + 2, m + 2, dp);
    _2dsegtree tr2(n + 2, m + 2, dp);
    _2dsegtree tr3(n + 2, m + 2, dp);
    _2dsegtree tr4(n + 2, m + 2, dp);
    vector<pair<int, int>> > type[p + 2];
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            int x;
            cin >> x;
            type[x].emplace_back(i, j);
        }
    }
    for (auto [x, y]: type[1]) {
        dp[x][y] = abs(x - 1) + abs(y - 1);
    }
    for (int i = 1; i < p; i++) {
        if (type[i].empty() or type[i + 1].empty()) continue;
        for (auto [x, y]: type[i]) {
            tr1.update(x, y, dp[x][y] - x - y);
            tr2.update(x, y, dp[x][y] - x + y);
            tr3.update(x, y, dp[x][y] + x - y);
            tr4.update(x, y, dp[x][y] + x + y);
        }
        for (auto [r, c]: type[i + 1]) {
            int ans1 = tr1.query(1, 1, r, c).val + r + c;
            int ans2 = tr2.query(1, c, r, m).val + r - c;
            int ans3 = tr3.query(r, 1, n, c).val - r + c;
            int ans4 = tr4.query(r, c, n, m).val - r - c;

```

```

        dp[r][c] = min({ans1, ans2, ans3, ans4});
    }
    for (auto [r, c]: type[i]) {
        tr1.update(r, c, 1e15);
        tr2.update(r, c, 1e15);
        tr3.update(r, c, 1e15);
        tr4.update(r, c, 1e15);
    }
}
if (!type[p].empty()) {
    auto [r, c] = type[p][0];
    cout << dp[r][c] << endl;
} else {
    cout << 0 << endl;
}
}

signed main() {
    wady
    files();
    int t = 1;
    int tc = 1;
    while (t--)
        solve(tc++);
}

```

2d sparsetable

```

struct SparseTable2D {
    int n, m;
    int max_log_n, max_log_m;
    vector<vector<vector<vector<int>>>> table;

    SparseTable2D(const vector<vector<int>>& grid) {
        n = grid.size();
        m = grid[0].size();
        max_log_n = std::bit_width(1ull * n);
        max_log_m = std::bit_width(1ull * m);

        table.assign(max_log_n, vector<vector<vector<int>>>(max_log_m,
            vector<vector<int>>(n, vector<int>(m))));

        // Base case: 1x1 blocks
    }
}

```

```

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            table[0][0][i][j] = grid[i][j];
        }
    }

    // Build for the first dimension (rows)
    for (int ir = 0; ir < n; ir++) {
        for (int jc = 1; jc < max_log_m; jc++) {
            for (int ic = 0; ic + (1 << jc) <= m; ic++) {
                table[0][jc][ir][ic] = merge(table[0][jc - 1][ir][ic],
                                                table[0][jc - 1][ir][ic + (1
<< (jc - 1))]));
            }
        }
    }

    // Build for the rest of the dimensions
    for (int jr = 1; jr < max_log_n; jr++) {
        for (int jc = 0; jc < max_log_m; jc++) {
            for (int ir = 0; ir + (1 << jr) <= n; ir++) {
                for (int ic = 0; ic + (1 << jc) <= m; ic++) {
                    table[jr][jc][ir][ic] = merge(table[jr - 1][jc][ir]
[ic],
                                                table[jr - 1][jc][ir +
(1 << (jr - 1))] [ic]);
                }
            }
        }
    }

    int merge(int a, int b) {
        return max(a, b);
    }

    int query(int r1, int c1, int r2, int c2) {
        int kr = std::bit_width(1ull * (r2 - r1 + 1)) - 1;
        int kc = std::bit_width(1ull * (c2 - c1 + 1)) - 1;

        int top_left = table[kr][kc][r1][c1];
        int bottom_left = table[kr][kc][r2 - (1 << kr) + 1][c1];
        int top_right = table[kr][kc][r1][c2 - (1 << kc) + 1];
        int bottom_right = table[kr][kc][r2 - (1 << kr) + 1][c2 - (1 << kc) +
1];

```

```

        return merge(merge(top_left, bottom_left), merge(top_right,
bottom_right));
    }
};

```

BIT

```

struct FenwickTree {
    vector<int> bit;
    int n;
    int LOGN;
    // 1 based
    FenwickTree(int sz) {
        n = sz + 1;
        LOGN = 0;
        while ((1 << LOGN) <= n) LOGN++;
        bit.assign(n, 0);
    }

    void add(int idx, int delta) {
        for (; idx < n; idx += idx & -idx)
            bit[idx] += delta;
    }

    int sum(int idx) {
        int ret = 0;
        for (; idx > 0; idx -= idx & -idx)
            ret += bit[idx];
        return ret;
    }

    int sum(int l, int r) {
        if (l > r) return 0;
        return sum(r) - sum(l - 1);
    }

    // Find the k-th order statistic (requires a 1-based tree)
    // Returns the 1-based index of the largest value less than or equal to v
    int bit_search(int v) {
        int sum = 0;
        int pos = 0;
        for (int i = LOGN; i >= 0; i--) {
            if (pos + (1 << i) < n && sum + bit[pos + (1 << i)] < v) {
                sum += bit[pos + (1 << i)];
                pos += (1 << i);
            }
        }
    }
};

```

```

        }
    }
    return pos + 1;
}
};

```

Range update

```

#include<bits/stdc++.h>
using namespace std;

const int N = 3e5 + 9;

struct BIT {
    long long M[N], A[N];
    BIT() {
        memset(M, 0, sizeof M);
        memset(A, 0, sizeof A);
    }
    void update(int i, long long mul, long long add) {
        while (i < N) {
            M[i] += mul;
            A[i] += add;
            i |= (i + 1);
        }
    }
    void upd(int l, int r, long long x) {
        update(l, x, -x * (l - 1));
        update(r, -x, x * r);
    }
    long long query(int i) {
        long long mul = 0, add = 0;
        int st = i;
        while (i >= 0) {
            mul += M[i];
            add += A[i];
            i = (i & (i + 1)) - 1;
        }
        return (mul * st + add);
    }
    long long query(int l, int r) {
        return query(r) - query(l - 1);
    }
} t;

```

```
int32_t main() {

    return 0;
}
```

Centriod Decompstion

```
const int N = 1e5 + 20;
int n, sz[N], par2[N];
bool done[N];
vector<int> adj[N], adj2[N]; //adj is original tree, adj2 is centroid tree

int update_sz(int node, int par) {
    sz[node] = 1;
    for (auto ch: adj[node]) {
        if (ch == par || done[ch]) continue;
        update_sz(ch, node);
        sz[node] += sz[ch];
    }
    return sz[node];
}

int get_centroid(int node, int par, int n) {
    int mx = 0;
    for (auto ch: adj[node]) {
        if (ch == par || done[ch]) continue;
        if (sz[ch] > sz[mx])
            mx = ch;
    }

    if (sz[mx] * 2 > n)
        return get_centroid(mx, node, n);
    else
        return node;
}

int decompose(int node, int par, int d) {
    int cen = get_centroid(node, par, update_sz(node, par));
    done[cen] = 1;
    if (par != node) {
        adj2[par].push_back(cen);
        par2[cen] = par;
    }

    for (auto ch: adj[cen]) {
```

```

        if (done[ch]) continue;
        decompose(ch, cen, d + 1);
    }

    return cen;
}

```

Compressor

```

struct Compressor {
    vector<int> vals;

    void add(int x) {
        vals.push_back(x);
    }

    void build() {
        sort(vals.begin(), vals.end());
        vals.erase(unique(vals.begin(), vals.end()), vals.end());
    }

    int get_id(int x) const {
        return lower_bound(vals.begin(), vals.end(), x) - vals.begin();
    }

    int get_val(int idx) const {
        return vals[idx];
    }

    int size() const {
        return vals.size();
    }
};

```

DSU

```

struct DSU {
    vector<int> sz, parent;

    DSU(int n) {
        sz.resize(n + 1, 1);
        parent.resize(n + 1);
        for (int i = 0; i <= n; i++) {
            parent[i] = i;
        }
    }
};

```

```

    }
}

int find(int node) {
    if (node == parent[node])
        return node;
    return parent[node] = find(parent[node]);
}

void unionset(int u, int v) {
    u = find(u);
    v = find(v);
    if (u == v) {
        return;
    }
    if (sz[u] < sz[v]) {
        parent[u] = v;
        sz[v] += sz[u];
    } else {
        parent[v] = u;
        sz[u] += sz[v];
    }
}
};

```

Roll back DSU

```

struct RollBackUnionFind {
    using elem_tp = tuple<int, int, int, int>;

    vector<int> node, size_vec;
    stack<elem_tp> info;

    RollBackUnionFind(int N) : node(N), size_vec(N, 1) {
        for (int i = 0; i < N; i++) node[i] = i;
    }

    int find(int u) {
        return node[u] == u ? u : find(node[u]);
    }

    int size(int u) {
        return size_vec[find(u)];
    }
}

```

```

bool unite(int u, int v) {
    u = find(u), v = find(v);
    info.emplace(u, node[u], v, node[v]);
    if (u == v) return false;

    if (size(u) > size(v)) swap(u, v);
    node[u] = v;
    size_vec[v] += size_vec[u];
    return true;
}

int same(int u, int v) {
    return find(u) == find(v);
}

void rollback() {
    assert(info.size());
    int u, nu, v, nv;
    tie(u, nu, v, nv) = info.top();
    info.pop();
    node[u] = nu, node[v] = nv;
}
};

```

persistent

```

struct persistent_dsu {
    struct state {
        int u, ru, v, rv;
        state() {
            u = 0;
            ru = 0;
            v = 0;
            rv = 0;
        }
        state(int _u, int _ru, int _v, int _rv) {
            u = _u;
            ru = _ru;
            v = _v;
            rv = _rv;
        }
    };
};

int cnt;
int depth[N], par[N];

```

```

stack<state> st;

persistent_dsu() {
    cnt = 0;
    memset(depth, 0, sizeof(depth));
    memset(par, 0, sizeof(par));
    while(!st.empty()) st.pop();
}

void init(int _sz) {
    cnt = _sz;
    for(int i = 0; i <= _sz; i++)
        par[i] = i, depth[i] = 1;
}

int root(int x) {
    if(x == par[x]) return x;
    return root(par[x]);
}

bool connected(int x, int y) {
    return root(x) == root(y);
}

void unite(int x, int y) {
    int rx = root(x), ry = root(y);
    if(rx == ry) return;

    if(depth[rx] < depth[ry])
        par[rx] = ry;
    else if(depth[ry] < depth[rx])
        par[ry] = rx;
    else par[rx] = ry, depth[ry]++;

    cnt--;
    st.push(state(rx, depth[rx], ry, depth[ry]));
}

void snapshot() {
    st.push(state(-1, -1, -1, -1));
}

void rollback() {
    while(!st.empty()) {
        if(st.top().u == -1)

```

```

        return;

        ++cnt;
        par[st.top().u] = st.top().u;
        par[st.top().v] = st.top().v;
        depth[st.top().u] = st.top().ru;
        depth[st.top().v] = st.top().rv;
        st.pop();
    }
}
};

```

DSU on tree

```

#include <bits/stdc++.h>
using namespace std;
#define ll long long
const int N=200005;
int n,c[N],sz[N],ans[N];vector<int>adj[N],subTree[N];map<int,int>m;
void dfs(int node,int par)
{
    sz[node]=1;
    for(auto it:adj[node])
    {
        if(it!=par)
        {
            dfs(it,node);
            sz[node]+=sz[it];
        }
    }
}
void sack(int node,int par,bool keep)
{
    // find big child
    int mx=-1,bigCh=-1;
    for(auto it:adj[node])
    {
        if(it!=par&&mx<sz[it])
        {
            mx=sz[it];
            bigCh=it;
        }
    }
    // go deep into small children

```

```

for(auto it:adj[node])
{
    if(it!=par&&it!=bigCh)
    {
        sack(it,node,0);
    }
}
// go deep into big child
if(~bigCh)
{
    sack(bigCh,node,1);
    swap(subTree[node],subTree[bigCh]);
}
// add information of node
subTree[node].push_back(node);
m[c[node]]++;
for(auto it:adj[node])
{
    if(it!=par&&it!=bigCh)
    {
        for(auto u:subTree[it])
        {
            m[c[u]]++;
            subTree[node].push_back(u);
        }
    }
}
ans[node]=m.size();
// small to large
if(keep==0)
{
    for(auto it:subTree[node])
    {
        m[c[it]]--;
        if(m[c[it]]==0) m.erase(m.find(c[it]));
    }
}
}
signed main()
{
    ios_base::sync_with_stdio(0);cin.tie(0);cout.tie(0);
    cin>>n;
    for(int i=1;i<=n;i++) cin>>c[i];
    for(int i=1;i<n;i++)
    {
        int a,b;cin>>a>>b;
    }
}

```

```

        adj[a].push_back(b);
        adj[b].push_back(a);
    }
    dfs(1,1);
    sack(1,1,0);
    for(int i=1;i<=n;i++) cout<<ans[i]<<' ';
    return 0;
}

```

Dynamic Segment Tree

```

#include <bits/stdc++.h>
using namespace std;

#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;
template<typename T>
using orderedset = tree<T, null_type, less<T>, rb_tree_tag,
tree_order_statistics_node_update>;
#define int long long
#define ll long long
#define ld long double
#define endl '\n'
#define wady \
    ios_base::sync_with_stdio(0); \
    cin.tie(0); \
    cout.tie(0);

void files() {
#ifdef ONLINE_JUDGE
    freopen("in.txt", "r", stdin);
    freopen("out.txt", "w", stdout);
#endif
}

struct implicitsegtree {
    struct Node {
        int freq = 0;
        int lazy = 0;
        int left = -1;

```

```

    int right = -1;
};

vector<Node> tree;
const int n;
int timer = 0;

int merge(int a, int b) { return a + b; }

void apply(int cur, int len, int val) {
    if (val == 1) {
        tree[cur].lazy = val;
        tree[cur].freq = len * val;
    }
}

void push_down(int cur, int l, int r) {
    if (tree[cur].left == -1) {
        tree[cur].left = ++timer;
        tree.push_back(Node());
    }
    if (tree[cur].right == -1) {
        tree[cur].right = ++timer;
        tree.push_back(Node());
    }
    int m = (l + r) / 2;
    apply(tree[cur].left, m - l + 1, tree[cur].lazy);
    apply(tree[cur].right, r - m, tree[cur].lazy);
    tree[cur].lazy = 0;
}

void update(int cur, int l, int r, int ql, int qr, int val) {
    if (qr < l || ql > r) { return; }
    if (ql <= l && r <= qr) {
        apply(cur, r - l + 1, val);
    } else {
        push_down(cur, l, r);
        int m = (l + r) / 2;
        update(tree[cur].left, l, m, ql, qr, val);
        update(tree[cur].right, m + 1, r, ql, qr, val);
        tree[cur].freq =
            merge(tree[tree[cur].left].freq,
tree[tree[cur].right].freq);
    }
}

```

```

int query(int cur, int l, int r, int ql, int qr) {
    if (qr < l || ql > r) { return 0; }
    if (ql <= l && r <= qr) { return tree[cur].freq; }
    push_down(cur, l, r);
    int m = (l + r) / 2;
    return merge(query(tree[cur].left, l, m, ql, qr),
                 query(tree[cur].right, m + 1, r, ql, qr));
}

implicitsegtree(int n, int q = 0) : n(n) {
    if (q > 0) { tree.reserve(2 * q * __lg(n)); }
    tree.push_back(Node());
}

void update(int ql, int qr, int val) { update(0, 0, n - 1, ql, qr, val); }

int query(int ql, int qr) { return query(0, 0, n - 1, ql, qr); }
};

void solve(int tc) {
    int q;
    cin >> q;
    implicitsegtree tr((int) 1e9 + 2, q);
    int prv = 0;
    while (q--) {
        int t, l, r;
        cin >> t >> l >> r;
        if (t == 1) {
            int ans = tr.query(l + prv, r + prv);
            cout << ans << endl;
            prv = ans;
        } else {
            tr.update(l + prv, r + prv, 1);
        }
    }
}

signed main() {
    wady
    files();
    int t = 1;
    int tc = 1;
    while (t--)
        solve(tc++);
}

```

HLD

```
const int N = 2e5 + 20;
int par[N], depth[N], heavy[N], head[N], pos[N];
int cur_pos = 0;
vector<int> adj[N];
int n;

struct segmenttree {
    int sz;
    vector<int> tree;

    segmenttree() {}

    segmenttree(int _n) {
        sz = _n;
        tree.assign(4 * sz, 0);
    }

    int merge(int node) {
        return max(tree[node * 2 + 1], tree[node * 2 + 2]);
    }

    void update(int idx, int val, int node = 0, int lx = 0, int rx = -1) {
        if (rx == -1) rx = sz - 1;

        if (lx == rx) {
            tree[node] = val;
            return;
        }

        int mid = (lx + rx) >> 1;
        if (idx <= mid)
            update(idx, val, node * 2 + 1, lx, mid);
        else
            update(idx, val, node * 2 + 2, mid + 1, rx);

        tree[node] = merge(node);
    }

    int query(int l, int r, int node = 0, int lx = 0, int rx = -1) {
        if (rx == -1) rx = sz - 1;

        if (lx > r || rx < l)
            return -1e18;
    }
}
```

```

        if (lx >= l && rx <= r)
            return tree[node];

        int mid = (lx + rx) >> 1;
        return max(
            query(l, r, node * 2 + 1, lx, mid),
            query(l, r, node * 2 + 2, mid + 1, rx)
        );
    }
}seg;

int dfs(int node) {
    int sz = 1;
    int max_chsz = 0;
    for (auto &child: adj[node]) {
        if (child == par[node])continue;
        par[child] = node;
        depth[child] = depth[node] + 1;
        int chsz = dfs(child);
        sz += chsz;
        if (chsz > max_chsz) {
            max_chsz = chsz;
            heavy[node] = child;
        }
    }
    return sz;
}

void decompse(int node, int h) {
    head[node] = h;
    pos[node] = cur_pos++;
    if (heavy[node] != -1) {
        decompse(heavy[node], h);
    }
    for (auto child: adj[node]) {
        if (child == par[node] or child == heavy[node])continue;
        decompse(child, child);
    }
}

void init() {
    fill(par, par + n + 5, 0);
    fill(depth, depth + n + 5, 0);
    fill(heavy, heavy + n + 5, -1);
}

```

```

fill(head, head + n + 5, 0);
fill(pos, pos + n + 5, 0);
cur_pos = 0;
dfs(0);
decompse(0, 0);
}

int query(int a, int b) {
    int ret = 0;
    for (; head[a] != head[b]; b = par[head[b]]) {
        if (depth[head[a]] > depth[head[b]]) {
            swap(a, b);
        }
        ret = max(ret, seg.query(pos[head[b]], pos[b]));
    }
    if (depth[a] > depth[b]) {
        swap(a, b);
    }
    // if edges should be pos[a] + 1
    ret = max(ret, seg.query(pos[a], pos[b]));
    return ret;
}

int lca(int u, int v) {
    while (head[u] != head[v]) {
        if (depth[head[u]] < depth[head[v]]) swap(u, v);
        u = par[head[u]];
    }
    if (depth[u] < depth[v]) return u;
    return v;
}

// edges
for (auto &[u, v, c]: edges) {
    if (depth[u] < depth[v]) swap(u, v);
    seg.update(pos[u], c);
}

```

Lazy Segment Tree

```

struct SegmentTree {
    struct Node {
        int f = -1e9;
    };
    Node skip;

```

```

vector<Node> tree;
vector<int> lazy;
int n;
SegmentTree() {}
SegmentTree(int m, vector<int>& a) {
    n = m;
    tree.resize(n << 2);
    lazy.resize(n << 2, 0);
    skip.f = -1e9; // value to skip when query
    build(1, 0, n - 1, a);
}

// base case
Node single(int x) {
    Node ret;
    ret.f = x;
    return ret;
}

Node merge(Node a, Node b) {
    Node ret;
    ret.f = max(a.f, b.f);
    return ret;
}

void build(int u, int st, int en, vector<int> &a) {
    if (st == en) {
        tree[u] = single(a[st]);
        return;
    }
    int mid = st + en >> 1;
    build(u << 1, st, mid, a);
    build((u << 1) | 1, mid + 1, en, a);
    tree[u] = merge(tree[u << 1], tree[(u << 1) | 1]);
}

void prop(int u, int st, int en) {
    if (!lazy[u]) return;

    // update the node value
    tree[u].f += lazy[u];

    // move the lazy to my children
    if (st != en) {
        lazy[u << 1] += lazy[u];
        lazy[(u << 1) | 1] += lazy[u];
    }

    lazy[u] = 0;
}

```

```

}
void update(int u, int st, int en, int idx, int delta) {
    prop(u, st, en);
    if (idx > en || idx < st) return;
    if (st == en) {
        tree[u].f += delta;
        return;
    }
    int mid = st + en >> 1;
    update(u << 1, st, mid, idx, delta);
    update((u << 1) | 1, mid + 1, en, idx, delta);
    tree[u] = merge(tree[u << 1], tree[(u << 1) | 1]);
}

void update_range(int u, int st, int en, int lx, int rx, int delta) {
    prop(u, st, en);
    if (st > rx || en < lx) return;
    if (st >= lx && en <= rx) {
        lazy[u] += delta;
        prop(u, st, en);
        return;
    }
    int mid = st + en >> 1;
    update_range(u << 1, st, mid, lx, rx, delta);
    update_range((u << 1) | 1, mid + 1, en, lx, rx, delta);
    tree[u] = merge(tree[u << 1], tree[(u << 1) | 1]);
}

Node query(int u, int st, int en, int lx, int rx) {
    prop(u, st, en);
    if (st > rx || en < lx) return skip;
    if (st >= lx && en <= rx) return tree[u];
    int mid = st + en >> 1;
    return merge(query(u << 1, st, mid, lx, rx),
        query((u << 1) | 1, mid + 1, en, lx, rx));
}

int query(int lx, int rx) {
    --lx, --rx;
    return query(1, 0, n - 1, lx, rx).f;
}

void update(int idx, int delta) {
    --idx;
    update(1, 0, n - 1, idx, delta);
}

void update_range(int lx, int rx, int delta) {
    --lx, --rx;
    update_range(1, 0, n - 1, lx, rx, delta);
}

```

```
}  
};
```

MO on tree

Subtree ranges

Path ranges

- Case 1 : u is ancestor of v
- Case 2 : and add LCA manually

```
#include <bits/stdc++.h>  
#include <ext/pb_ds/assoc_container.hpp>  
#include <ext/pb_ds/tree_policy.hpp>  
  
using namespace std;  
using namespace __gnu_pbds;  
  
template<typename T>  
using orderedset = tree<T, null_type, less<T>, rb_tree_tag,  
tree_order_statistics_node_update>;  
  
#define int long long  
#define ll long long  
#define ld long double  
#define endl '\n'  
#define wady \\\n    ios_base::sync_with_stdio(0); \\\n    cin.tie(0); \\\n    cout.tie(0);  
  
void files() {  
#ifndef ONLINE_JUDGE  
    freopen("in.txt", "r", stdin);  
    freopen("out.txt", "w", stdout);  
#endif  
}  
  
const int N = 200005, M = 1e6 + 20, LG = 20;  
int SQ;  
vector<int> adj[N];  
int in[N], out[N], lvl[N];  
int flat[2 * N], a[N], ans[M];  
int up[N][LG];
```

```

int timer = 0;
int n, q;
bool seen[N];
int frq[N];
int cnt = 0;

void dfs(int node, int par) {
    in[node] = ++timer;
    up[node][0] = par;
    flat[timer] = node;
    lvl[node] = lvl[par] + 1;
    for (int i = 1; i < LG; i++) {
        up[node][i] = up[up[node][i - 1]][i - 1];
    }
    for (auto child: adj[node]) {
        if (child == par) continue;
        dfs(child, node);
    }
    out[node] = ++timer;
    flat[timer] = node;
}

struct query {
    int l, r, lc, idx;

    query(int _l, int _r, int _lc, int _idx) {
        l = _l;
        r = _r;
        lc = _lc;
        idx = _idx;
    }

    bool operator<(const query &oth) const {
        int b1 = l / SQ, b2 = oth.l / SQ;
        if (b1 != b2) return b1 < b2;
        return (b1 & 1) ? r < oth.r : r > oth.r;
    }
};

int LCA(int x, int y) {
    if (lvl[x] > lvl[y]) { swap(x, y); }
    int diff = lvl[y] - lvl[x];
    for (int i = 0; (1 << i) <= diff; i++) {
        if ((1 << i) & diff) { y = up[y][i]; }
    }
    if (x == y) { return x; }
}

```

```

    for (int i = LG - 1; i >= 0; i--) {
        if (up[x][i] != up[y][i]) {
            x = up[x][i];
            y = up[y][i];
        }
    }
    return up[x][0];
}

void check(int node) {
    if (seen[node]) {
        frq[a[node]]--;
        if (frq[a[node]] == 0) cnt--;
    } else {
        if (frq[a[node]] == 0) cnt++;
        frq[a[node]]++;
    }
    seen[node] ^= 1;
}

void MO(vector<query> &queries) {
    int l = 1, r = 0;
    for (auto [ql, qr, lc, idx]: queries) {
        while (l > ql) check(flat[--l]);
        while (r < qr) check(flat[++r]);
        while (l < ql) check(flat[l++]);
        while (r > qr) check(flat[r--]);
        if (lc != -1) check(lc);
        ans[idx] = cnt;
        if (lc != -1) check(lc);
    }
}

void solve(int tc) {
    cin >> n >> q;

    timer = 0;
    cnt = 0;
    for (int i = 0; i <= n; i++) {
        adj[i].clear();
        seen[i] = false;
        frq[i] = 0;
    }

    vector<int> vals;
    for (int i = 1; i <= n; i++) {

```

```

        cin >> a[i];
        vals.push_back(a[i]);
    }

    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());
    for (int i = 1; i <= n; i++) {
        a[i] = lower_bound(vals.begin(), vals.end(), a[i]) - vals.begin() + 1;
    }

    int m = n - 1;
    SQ = max(1LL, (long long)sqrt(2 * n + 2));
    while (m--) {
        int u, v;
        cin >> u >> v;
        adj[u].emplace_back(v);
        adj[v].emplace_back(u);
    }

    dfs(1, 1);

    vector<query> queries;
    for (int i = 1; i <= q; i++) {
        int u, v;
        cin >> u >> v;
        if (in[u] > in[v]) swap(u, v);
        int lc = LCA(u, v);
        if (lc == u) {
            queries.emplace_back(in[u], in[v], -1, i);
        } else {
            queries.emplace_back(out[u], in[v], lc, i);
        }
    }

    sort(queries.begin(), queries.end());
    MO(queries);

    for (int i = 1; i <= q; i++) {
        cout << ans[i] << endl;
    }
}

signed main() {
    wady
    files();
    int t = 1;

```

```

    int tc = 1;
    while (t--)
        solve(tc++);
}

```

MO with dsu

```

#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;
template<typename T>
using orderedset = tree<T, null_type, less<T>, rb_tree_tag,
tree_order_statistics_node_update>;
#define int long long
#define ll long long
#define ld long double
#define endl '\n'
#define wady \
    ios_base::sync_with_stdio(0); \
    cin.tie(0); \
    cout.tie(0);

void files() {
#ifdef ONLINE_JUDGE
    freopen("in.txt", "r", stdin);
    freopen("out.txt", "w", stdout);
#endif
}

const int N = 2e5 + 20;

struct persistent_dsu {
    struct state {
        int u, ru, v, rv;

        state(int _u, int _ru, int _v, int _rv) : u(_u), ru(_ru), v(_v),
rv(_rv) {
        }
    };

    int cnt;

```

```

int depth[N], par[N];
stack<state> st;

void init(int _sz) {
    cnt = _sz;
    for (int i = 0; i <= _sz; i++) {
        par[i] = i;
        depth[i] = 1;
    }
    while (!st.empty()) st.pop();
}

int root(int x) {
    while (x != par[x]) x = par[x];
    return x;
}

void unite(int x, int y) {
    int rx = root(x), ry = root(y);
    if (rx == ry) return;
    if (depth[rx] < depth[ry]) swap(rx, ry);
    st.push(state(ry, depth[ry], rx, depth[rx]));
    par[ry] = rx;
    if (depth[rx] == depth[ry]) depth[rx]++;
    cnt--;
}

void snapshot() {
    st.push(state(-1, -1, -1, -1));
}

void rollback() {
    while (!st.empty() && st.top().u != -1) {
        state s = st.top();
        st.pop();
        par[s.u] = s.u;
        depth[s.u] = s.ru;
        par[s.v] = s.v;
        depth[s.v] = s.rv;
        cnt++;
    }
    if (!st.empty()) st.pop();
}
} dsu;

struct query {

```

```

    int l, r, idx;

    query(int _l, int _r, int _idx) : l(_l), r(_r), idx(_idx) {
    }
};

int sq;

bool cmp(const query &f, const query &s) {
    int bf = f.l / sq;
    int bs = s.l / sq;
    if (bf != bs) return bf < bs;
    return f.r < s.r;
}

void solve(int tc) {
    int n, m, q;
    cin >> n >> m;

    sq = max(1LL, (int) sqrt(m));
    vector<pair<int, int> > edges(m + 1);
    for (int i = 1; i <= m; i++) cin >> edges[i].first >> edges[i].second;
    cin >> q;
    vector<query> queries;
    vector<int> ans(q + 1);

    for (int i = 1; i <= q; i++) {
        int l, r;
        cin >> l >> r;
        if (l / sq == r / sq) {
            dsu.init(n);
            for (int j = l; j <= r; j++) {
                auto [u, v] = edges[j];
                dsu.unite(u, v);
            }
            ans[i] = dsu.cnt;
        } else {
            queries.emplace_back(l, r, i);
        }
    }

    sort(queries.begin(), queries.end(), cmp);

    int lstblk = -1, border = 0, r_ptr = 0;

    for (auto &qu: queries) {

```

```

    int cur_blk = qu.l / sq;
    if (cur_blk != lstblk) {
        dsu.init(n);
        border = (cur_blk + 1) * sq;
        r_ptr = border;
        lstblk = cur_blk;
    }

    while (r_ptr < qu.r) {
        r_ptr++;
        dsu.unite(edges[r_ptr].first, edges[r_ptr].second);
    }

    dsu.snapshot();
    for (int k = qu.l; k <= border; k++) {
        dsu.unite(edges[k].first, edges[k].second);
    }
    ans[qu.idx] = dsu.cnt;
    dsu.rollback();
}

for (int i = 1; i <= q; i++) cout << ans[i] << endl;
}

signed main() {
    wady
    files();
    int t = 1;
    int tc = 1;
    while (t--)
        solve(tc++);
}

```

Parallel DSU

```

#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;
template<typename T>
using orderedset = tree<T, null_type, less<T>, rb_tree_tag,

```

```

tree_order_statistics_node_update>;
#define int long long
#define ll long long
#define ld long double
#define endl '\n'
#define wady \
    ios_base::sync_with_stdio(0); \
    cin.tie(0); \
    cout.tie(0);

void files() {
#ifdef ONLINE_JUDGE
    freopen("in.txt", "r", stdin);
    freopen("out.txt", "w", stdout);
#endif
}

struct DSU {
    vector<int> sz, parent;

    DSU(int n) {
        sz.resize(n + 1, 1);
        parent.resize(n + 1);
        for (int i = 0; i <= n; i++) {
            parent[i] = i;
        }
    }

    int find(int node) {
        if (node == parent[node])
            return node;
        return parent[node] = find(parent[node]);
    }

    void unionset(int u, int v) {
        u = find(u);
        v = find(v);
        if (u == v) {
            return;
        }
        if (sz[u] < sz[v]) {
            parent[u] = v;
            sz[v] += sz[u];
        } else {
            parent[v] = u;
            sz[u] += sz[v];
        }
    }
};

```

```

    }
}
};

const int B = 20;

void solve(int tc) {
    int n, m;
    cin >> n >> m;
    vector<DSU> dsus(B, DSU(n + 2));
    while (m--) {
        int i1, j1, i2, j2;
        cin >> i1 >> j1 >> i2 >> j2;
        int len = j1 - i1 + 1;
        for (int i = B - 1; i >= 0; i--) {
            if (len & (1ll << i)) {
                dsus[i].unionset(i1, i2);
                i1 += (1ll << i);
                i2 += (1ll << i);
            }
        }
    }
    for (int j = B - 1; j >= 1; j--) {
        for (int i = 1; i <= n; i++) {
            int par = dsus[j].find(i);
            if (par == i) continue;
            dsus[j - 1].unionset(i, par);
            int offset = (1ll << (j - 1));
            dsus[j - 1].unionset(i + offset, par + offset);
        }
    }
    set<int> st;
    for (int i = 1; i <= n; i++) {
        st.insert(dsus[0].find(i));
    }
    cout << st.size() << endl;
}

```

```

signed main() {
    wady
    files();
    int t = 1;
    int tc = 1;
    while (t--)

```

```

        solve(tc++);
    }

```

persistent Segment tree

```

// don't forget to Build()
// modify update & query for your problem
struct Node {
    int s;
    Node *l = NULL, *r = NULL;
    Node(int s = 0, Node* l = NULL, Node *r = NULL) : s(s), l(l), r(r) {}
};

Node* L(Node *node) { return node->l; }
Node* R(Node *node) { return node->r; }
int S(Node *node) { return node->s; }

Node *leaf(int s) { return new Node(s); }
Node *merge(Node *l, Node *r) {
    Node *ret = new Node();
    ret->l = l, ret->r = r;
    ret->s = S(l) + S(r);
    return ret;
}

Node *build(int l, int r) {
    if (l == r) return leaf(0);
    int md = (l + r) / 2;
    return merge(build(l, md), build(md + 1, r));
}

Node *update(Node* node, int l, int r, int idx, int s) {
    if (l == r) return leaf(s + node->s);
    int md = (l + r) / 2;
    if (idx <= md) return merge(update(L(node), l, md, idx, s), R(node));
    return merge(L(node), update(R(node), md + 1, r, idx, s));
}

int query(Node *node, int l, int r, int s, int e) {
    if (l > e || r < s) return 0;
    if (l >= s && r <= e) return S(node);
    int md = (l + r) / 2;
    return query(L(node), l, md, s, e) + query(R(node), md + 1, r, s, e);
}

```

implicit

```

// modify update & query for your problem
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
#define rep(i , st , ed) for(int i = st; i < ed; i++)
#define f first
#define s second
#define all(v) v.begin() , v.end()
#ifdef ONLINE_JUDGE
#define debug(x) cerr << #x << ": " << x << '\n';
#else
#define debug(x)
#endif
const int N = 1e9+1;

struct Node{
    Node* l, *r;
    ll s;
    Node(ll s = 0, Node* l = NULL, Node* r = NULL):
        s(s) , l(l) , r(r){}
};

Node* getL(Node* x){ return x == NULL ? x : x->l; }
Node* getR(Node* x){ return x == NULL ? x : x->r; }
ll getS(Node* x){ return x == NULL ? 0 : x->s; }
Node* newLeaf(int val){ return new Node(val); }

Node* newPar(Node* l, Node* r){
    Node* res = new Node();
    res->l = l; res->r = r;
    res->s = getS(l) + getS(r);
    return res;
}

int n; // n -> length of array
Node* upd(int i , int v , Node* x , int l = -N , int r = N){
    if(r - l == 1) return newLeaf( v + getS(x) );
    int m = (l+r)/2;
    if(i < m){
        return newPar(upd(i,v,getL(x),l,m) , getR(x));
    }else{
        return newPar(getL(x) , upd(i,v,getR(x),m,r));
    }
}

```

```

ll qry(int lx , int rx , Node* cur, Node* prv , int l = -N , int r = N){
    if(l >= lx && r <= rx) return (getS(cur) - getS(prv));
    if(r <= lx || l >= rx) return 0;
    int m = (l+r)/2;
    return qry(lx,rx,getL(cur), getL(prv) , l,m) + qry(lx,rx,getR(cur),
getR(prv),m,r);
}

int kth(int k , Node*prv , Node* cur, int l = -N , int r = N){
    if(r-l==1) return l;
    int m = (l+r)/2;

    int sumL = getS( getL(cur) ) - getS( getL(prv) );
    if(sumL >= k) return kth(k , getL(prv) , getL(cur) , l , m);
    return kth(k-sumL , getR(prv) , getR(cur) , m , r);
}

int main(){
    ios::sync_with_stdio(0); cin.tie(NULL); cout.tie(0);
    #ifndef ONLINE_JUDGE
    freopen("in.txt", "r", stdin);
    freopen("out.txt", "w", stdout);
    freopen("error.txt", "w", stderr);
    #endif

    int n , q; cin >> n >> q;
    vector<Node* > versions;

    versions.emplace_back(nullptr);
    for(int i = 1; i <= n; ++i){
        int x; cin >> x;
        versions.emplace_back( upd(x,+1 , versions.back()) );
    }

    while(q--){
        int i,j,k; cin >> i >> j >> k;
        cout << kth(k , versions[i-1] , versions[j]) << '\n';
    }
}

```

Segment Tree

```

struct SegmentTree {
    int n;
    struct Node {
        int val;
    };
    Node skip;
    vector<Node> tree;
    SegmentTree(int n, vector<int>& v) {
        this->n = n;
        tree.resize(n << 2);
        build(1, 0, n - 1, v);
    }
    Node single(int x) {
        Node r;
        r.val = x;
        return r;
    }
    Node merge(Node a, Node b) {
        Node ret;
        ret.val = a.val + b.val;
        return ret;
    }
    void build(int u, int st, int en, vector<int>& a) {
        if (st == en) return void(tree[u] = single(a[st]));
        int mid = st + en >> 1;
        build(u << 1, st, mid, a);
        build((u << 1) | 1, mid + 1, en, a);
        tree[u] = merge(tree[u << 1], tree[(u << 1) | 1]);
    }
    void update(int u, int st, int en, int idx, int val) {
        if (idx > en || idx < st) return;
        if (st == en) return void(tree[u] = single(val));
        int mid = st + en >> 1;
        update(u << 1, st, mid, idx, val);
        update((u << 1) | 1, mid + 1, en, idx, val);
        tree[u] = merge(tree[u << 1], tree[(u << 1) | 1]);
    }
    Node query(int u, int st, int en, int l, int r) {
        if (st >= l && en <= r) return tree[u];
        if (st > r || en < l) return skip;
        int mid = st + en >> 1;
        return merge(query(u << 1, st, mid, l, r), query((u << 1) | 1, mid + 1, en,
l, r));
    }
    void update(int idx, int val) {
        --idx;
    }
}

```

```

        update(1, 0, n - 1, idx, val);
    }
    Node query(int l, int r) {
        --l, --r;
        return query(1, 0, n - 1, l, r);
    }
};

```

Iterative Segment tree

```

const int N = 3e5 + 9;

int n, a[N];
struct Node {
    int sum;

    Node() { sum = 0; }
    Node(int val) { sum = val; }
};

struct ST {
    int n;
    Node *t;
    ST(int _n) { n = _n; t = new Node[2 * n]; }

    inline Node combine(Node l, Node r) {
        Node res;
        res.sum = l.sum + r.sum;
        return res;
    }

    void build() {
        for(int i = 0; i < n; i++) t[i + n] = Node(a[i+1]);
        for(int i = n - 1; i > 0; --i) t[i] = combine(t[i << 1], t[i << 1 | 1]);
    }

    void upd(int p, int v) {
        p--;
        for (t[p += n] = Node(v); p >= 1; ) t[p] = combine(t[p << 1], t[p << 1 | 1]);
    }

    Node query(int l, int r) {
        --l;
        bool f1 = 1, f2 = 1;
    }
};

```

```

Node resl, resr;
for(l += n, r += n; l < r; l >>= 1, r >>= 1) {
    if(l & 1) resl = f1 ? t[l++] : combine(resl, t[l++]), f1 = 0;
    if(r & 1) resr = f2 ? t[--r] : combine(t[--r], resr), f2 = 0;
}
if(f2) return resl;
if(f1) return resr;
return combine(resl, resr);
}
};

```

Sparse Table

```

struct SparseTable {
    vector<int> lg;
    vector<vector<int>> table;

    SparseTable(vector<int> &a) {
        int n = a.size();
        table = vector<vector<int>>(n + 5, vector<int>(20));
        lg.resize(n + 5);
        for (int i = 2; i <= n; i++)
            lg[i] = lg[i >> 1] + 1;
        for (int i = 0; i < n; i++)
            table[i][0] = a[i];
        for (int j = 1; j <= lg[n]; j++) {
            for (int i = 0; i + (1 << j) - 1 < n; i++)
                table[i][j] = merge(table[i][j - 1], table[i + (1 << (j - 1))][j - 1]);
        }
    }

    int query(int l, int r) {
        int sz = lg[r - l + 1];
        return merge(table[l][sz], table[r - (1 << sz) + 1][sz]);
    }

    int merge(int a, int b) { return gcd(a, b); }
};

```

SQRT and MO

```

const int N = 1e6 + 10, SQ = 175;
int n, q, ans, k;
int a[N], frq[N];

```

```

struct query {
    int l, r, i;

    bool operator<(query q) {
        int b1 = l / SQ, b2 = q.l / SQ;
        if (b1 != b2) return b1 < b2;
        return (b1 & 1) ? (r < q.r) : (r > q.r);
    }
};

void add(int idx) {
    frq[a[idx]]++;
    if(frq[a[idx]] == 1)ans++;
}

void rem(int idx) {
    frq[a[idx]]--;
    if(!frq[a[idx]])ans--;
}

vector<int> M0algo(vector<query> qq) {
    vector<int> ret(q);
    int l = 0, r = -1;

    for (auto &[ql, qr, qi]: qq) {
        while (r < qr) add(++r);
        while (l > ql) add(--l);
        while (r > qr) rem(r--);
        while (l < ql) rem(l++);
        ret[qi] = ans;
    }
    return ret;
}

const int Sq = 450, N = 2e5 + 10;
short a[N];
vector<pair<short, short>> lazy[N / Sq + 10];
int n, q;
short up[105];

void prog(int idx) {
    for (int i = 1; i <= 100; i++)up[i] = i;
    for (int i = lazy[idx].size() - 1; i >= 0; i--) {
        auto &[x, y] = lazy[idx][i];
        up[x] = up[y];
    }
}

```

```

    for (int i = idx * Sq; i < min(n, idx * Sq + Sq); i++) a[i] = up[a[i]];
    lazy[idx].clear();
}

void update(int l, int r, int x, int y) {
    prog(l / Sq);
    prog(r / Sq);
    for (int i = l; i <= r; i++) {
        if (i % Sq == 0 and i + Sq - 1 <= r) {
            lazy[i / Sq].emplace_back(x, y);
            i += Sq;
        } else {
            if (a[i] == x) a[i] = y;
            i++;
        }
    }
}

const int Sq = 450, N = 2e5 + 10;
vector<int> a(N), buc(N / Sq + 10);
int n, q;

void build(int idx) {
    buc[idx] = 0;
    for (int i = idx * Sq; i < min(n, (idx + 1) * Sq); i++) {
        buc[idx] += a[i];
    }
}

int query(int l, int r) {
    int ret = 0;
    for (int i = l; i <= r; i++) {
        if (i % Sq == 0 and i + Sq - 1 <= r) {
            ret += buc[i / Sq];
            i += Sq;
        } else {
            ret += a[i];
            i++;
        }
    }
    return ret;
}

```